



Tecnológico de Monterrey

Estudiante:

Elian Cantalapiedra Sabugal | A01738462

Profesor:

Alfredo García Suárez

Nombre de la Institución:

Instituto Tecnológico y de Estudios Superiores de
Tecnológico de Monterrey

Materia:

Implementación de la Robótica Inteligente

Título del trabajo:

Actividad 1.7 (Waypoints)

Metodología

- Extracción de waypoints: se identifican las coordenadas (x, y) de cada vértice de la figura sobre la cuadrícula y se ordenan según el recorrido en la matriz waypoints.
- Modelado del vehículo: se define un robot de tracción diferencial con su radio de rueda R y distancia entre ruedas L mediante DifferentialDrive.
- Parámetros de simulación: se fija el tiempo de muestreo sampleTime, se construye el vector de tiempo tVec (dimensionado a partir de la longitud de la ruta y la velocidad deseada) y se establece la pose inicial initPose = [x; y; θ] en el primer waypoint, orientada hacia el segundo.
- Controlador Pure Pursuit: se cargan los waypoints y se ajustan la distancia de anticipación (LookaheadDistance), la velocidad lineal deseada y la velocidad angular máxima, que determinan qué tan fiel sigue la trayectoria.
- Bucle de simulación: en cada paso el controlador entrega las velocidades de referencia (lineal y angular); la cinemática inversa las convierte en velocidades de rueda y la directa de nuevo en velocidades del cuerpo, que se transforman al marco global y se integran (Euler) para actualizar la pose.
- Visualización: con Visualizer2D se grafica en tiempo real el robot y los waypoints para observar el seguimiento.
- Validación y ajuste: se compara la trayectoria obtenida con la figura objetivo y se afinan la anticipación y las velocidades para corregir esquinas redondeadas o desviaciones.

Código

```
%% Definicion del vehiculo
R = 0.1;                % Radio de rueda [m]
L = 0.5;                % Distancia entre ruedas [m]
dd = DifferentialDrive(R,L);

% --- Grafica 1
% waypoints = [ 4, 4;    % A
%              -9, 8;    % B
%              7, -1;    % C
%              -9, -5;   % D
%              -1, 5;    % E
%              -3, 0;    % F
%              3, -5;    % G
%              0, 0];    % H
```

```

% --- Grafica 2
% waypoints = [ 2, 5;      % A
%              -5, 3;      % B
%              -5,-2;      % C
%              2,-5;       % D
%              5, 2;       % E
%              -3, 2;      % F
%              -4,-4;      % G
%              4,-3];     % H

% --- Grafica 3
waypoints = [-3, 4;      % A
            3, 3;       % B
            2,-3;       % C
            -1,-1;      % D
            1, 4;       % E
            -2,-4;      % F
            2,-1];     % G

%% Parametros de simulacion
sampleTime = 0.1;      % Tiempo de
muestreo [s]

desiredV = 1.0;        % Velocidad
lineal deseada [m/s]
pathLen = sum(vecnorm(diff(waypoints),2,2)); % Longitud
total de la trayectoria
Tsim = 1.4*pathLen/desiredV; % Tiempo total
(+40% de margen)
tVec = 0:sampleTime:Tsim; % Vector de
tiempo

% Pose inicial: en el primer waypoint, orientada hacia el
segundo
theta0 = atan2(waypoints(2,2)-waypoints(1,2), ...
              waypoints(2,1)-waypoints(1,1));
initPose = [waypoints(1,1); waypoints(1,2); theta0]; % [x;
y; theta]

pose = zeros(3,numel(tVec)); % Matriz de
poses
pose(:,1) = initPose;

%% Visualizador
viz = Visualizer2D;
viz.hasWaypoints = true;

%% Controlador Pure Pursuit

```

```

controller = controllerPurePursuit;
controller.Waypoints = waypoints;
controller.LookaheadDistance = 0.5;           % sube si
oscila, baja si corta curvas
controller.DesiredLinearVelocity = desiredV;
controller.MaxAngularVelocity = 2.0;          % mas alto =
giros mas cerrados

%% Lazo de simulacion
close all
r = rateControl(1/sampleTime);
for idx = 2:numel(tVec)
    % Pure Pursuit -> velocidades de referencia
    [vRef,wRef] = controller(pose(:,idx-1));

    % Cinematica inversa -> velocidades de rueda
    [wL,wR] = inverseKinematics(dd,vRef,wRef);

    % Cinematica directa -> velocidades del cuerpo
    [v,w] = forwardKinematics(dd,wL,wR);
    velB = [v;0;w];                               % [vx; vy; w]
en el cuerpo
    vel = bodyToWorld(velB,pose(:,idx-1));          %

    % Integracion discreta (Euler)
    pose(:,idx) = pose(:,idx-1) + vel*sampleTime;

    % Visualizacion
    viz(pose(:,idx),waypoints)
    waitfor(r);
end

```

Código figuras (parte 2)

```

%% Definicion del vehiculo
R = 0.1;           % Radio de rueda [m]
L = 0.5;           % Distancia entre ruedas [m]
dd = DifferentialDrive(R,L);

% ===== FIGURA 1 : PERRO=====
% waypoints = [ 5,12;  3,10;  1, 9;  1, 8;  2, 6;  3, 7;  4,
7; ...
%           5, 6;  6, 5;  6, 4;  7, 0; ...
%           6, 5; 11, 6; 12, 6; ...
%           6, 4; 10, 6; ...
%           11, 6;  8, 8;  7,12; ...
%           3,10;  4, 9;  4,10];

```

```

% ===== FIGURA 2 : ESTRELLA =====
% waypoints = [ 0, 5; 2, 7; 0, 9; 2,11; 4, 9; 6,11; 8,
9; 6, 7; 8, 5; ...
%             3, 6; 5, 6; 5, 8; 3, 8; 3, 6; ...
%             0, 5; 2, 5; 2, 3; 0, 3; 0, 5; ...
%             8, 5; 6, 5; 6, 3; 8, 3; 8, 5; ...
%             2, 3; 4, 5; 6, 3; 4, 1; 2, 3; ...
%             4, 5; 4, 1; ...
%             0, 3; 2, 1; 4, 1; ...
%             8, 3; 6, 1; 4, 1];

% ===== FIGURA 3 =====
waypoints = [ 3, 5; 3, 3; 5, 1; 7, 1; 9, 3; 9, 5; 7, 7;
5, 7; 3, 5; ...
            4, 9; 6,11; 8,11; 10, 9; 4, 9; ...
            10, 9; 6, 5; ...
            5, 6; 6, 5; 7, 5];

%
-----
-----

%% Parametros de simulacion
sampleTime = 0.1; % Tiempo de
muestreo [s]

desiredV = 1.0; % Velocidad
lineal deseada [m/s]
pathLen = sum(vecnorm(diff(waypoints),2,2)); % Longitud
total del recorrido
Tsim = 1.3*pathLen/desiredV; % Tiempo total
(+30% de margen)
tVec = 0:sampleTime:Tsim; % Vector de
tiempo

theta0 = atan2(waypoints(2,2)-waypoints(1,2), ...
               waypoints(2,1)-waypoints(1,1));
initPose = [waypoints(1,1); waypoints(1,2); theta0]; % [x;
y; theta]

pose = zeros(3,numel(tVec)); % Matriz de
poses
pose(:,1) = initPose;

%% Visualizador
viz = Visualizer2D;
viz.hasWaypoints = true;

%% Controlador Pure Pursuit

```

```

controller = controllerPurePursuit;
controller.Waypoints = waypoints;
controller.LookaheadDistance = 0.4;
controller.DesiredLinearVelocity = desiredV;
controller.MaxAngularVelocity = 2.5;

%% Lazo de simulacion
close all
r = rateControl(1/sampleTime);
for idx = 2:numel(tVec)
    % Pure Pursuit -> velocidades de referencia
    [vRef,wRef] = controller(pose(:,idx-1));

    % Cinematica inversa -> velocidades de rueda
    [wL,wR] = inverseKinematics(dd,vRef,wRef);

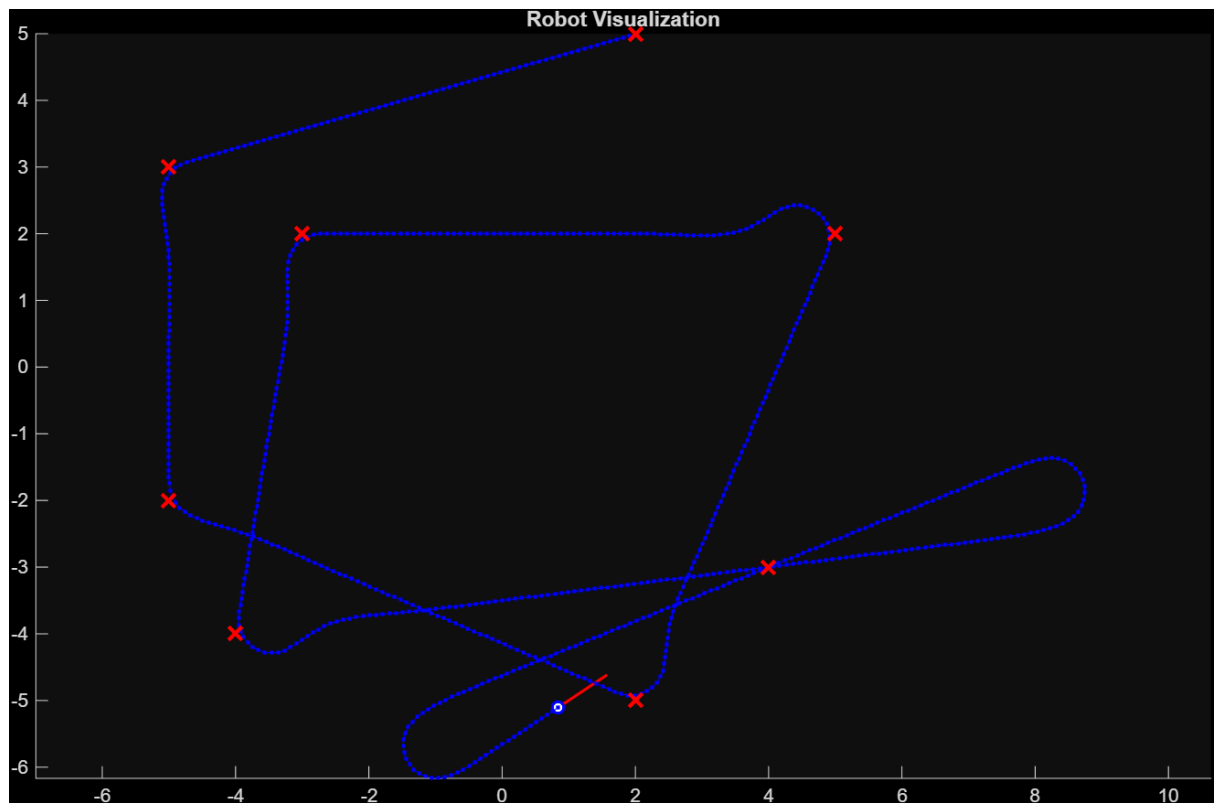
    % Cinematica directa -> velocidades del cuerpo
    [v,w] = forwardKinematics(dd,wL,wR);
    velB = [v;0;w];
    vel = bodyToWorld(velB,pose(:,idx-1));

    % Integracion discreta (Euler)
    pose(:,idx) = pose(:,idx-1) + vel*sampleTime;

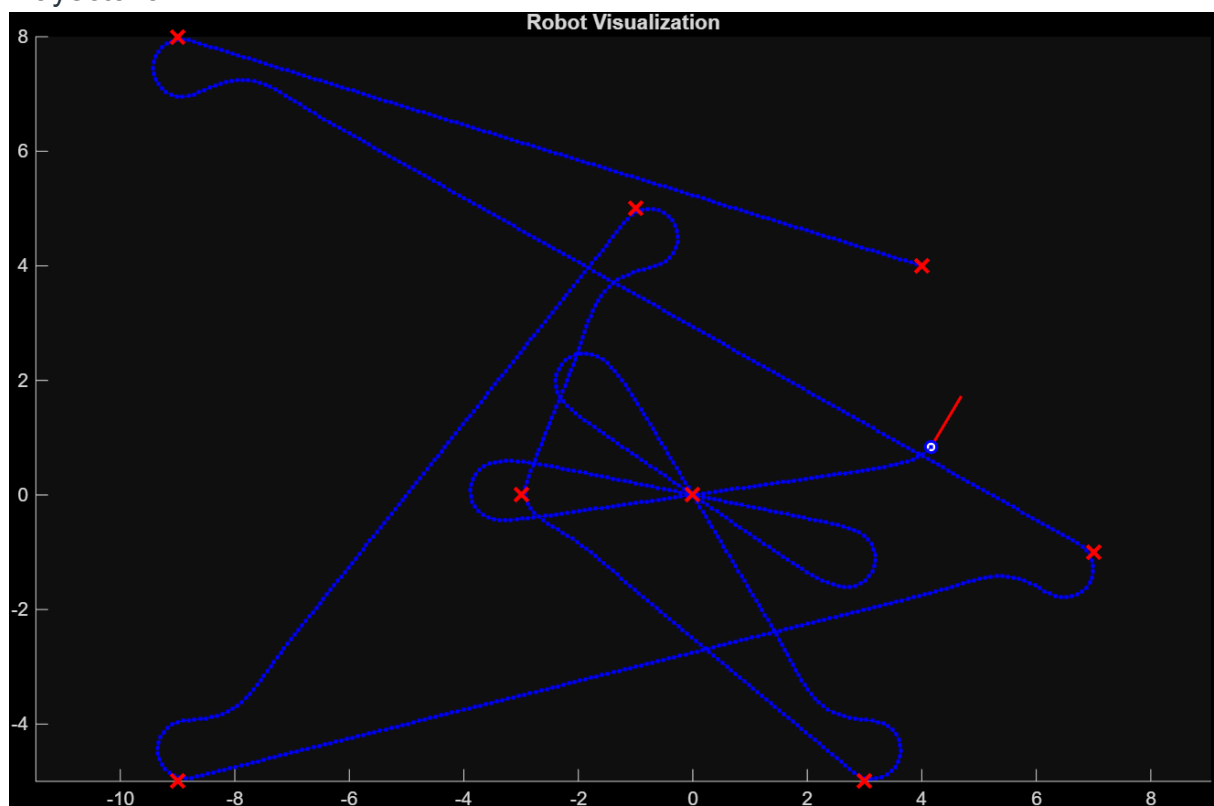
    % Visualizacion
    viz(pose(:,idx),waypoints)
    waitfor(r);
end

```

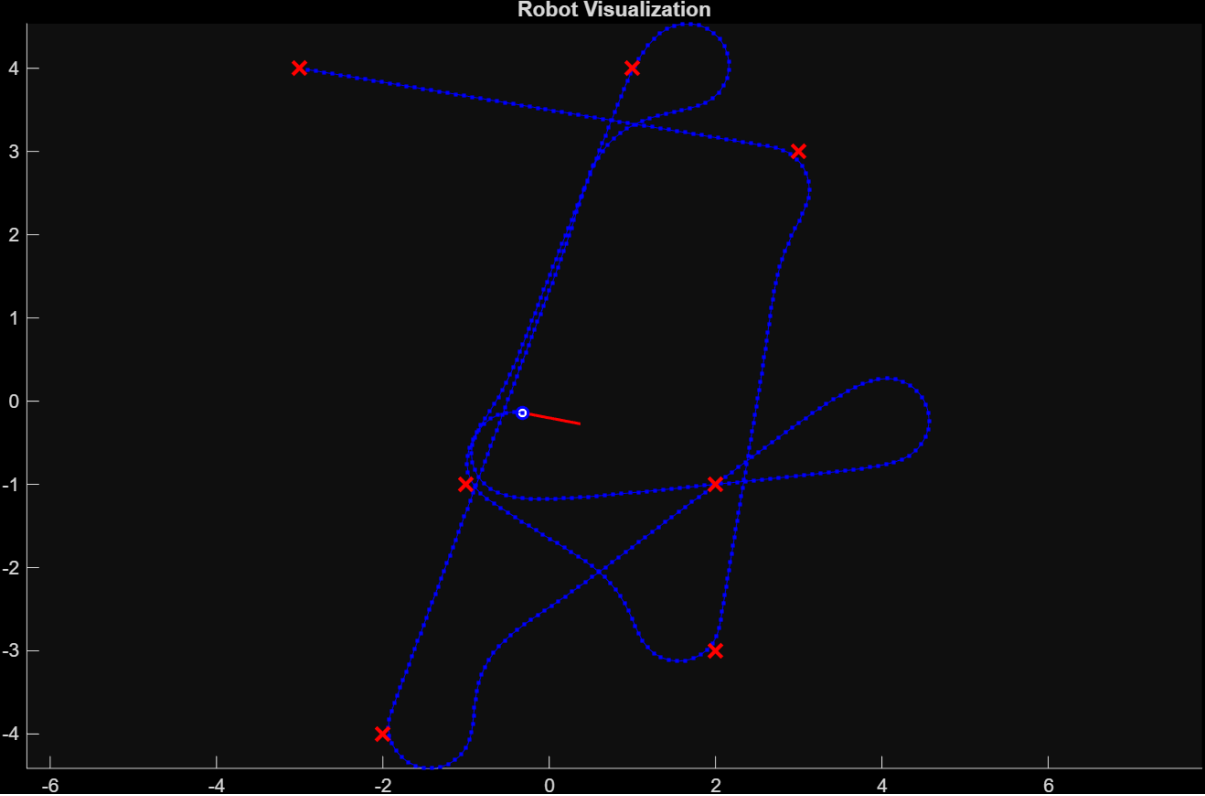
Resultados:



Trayectoria 1



Trayectoria 2



Trayectoria 3

Parte 2

